

we don't have the same notion of a dual between these LPs in non-zero-sum games.

In 1951, John Nash showed that equilibria always exist, even with > 2 players, and with games that are not zero sum. This utilizes something called Brouwer's fixed point theorem, which is a theorem in topology that is beyond the scope of this course.

In 2006, Daskalakis, Goldberg, and Papadimitriou showed that finding equilibria in even 3-player games is hard. It's not NP-hard (we haven't covered what this exactly means in the class yet), but it's a class of problems called PPAD-hard. Another group of researchers showed the very next year that it's even hard in 2-player games in general.

This means that even though Nash showed that equilibria always exist, it's computationally hard to actually solve for that equilibrium.

10/26/2021

Lecture 17

Multiplicative Weights

17.1 Online Decision-making

Today, we'll talk about an algorithm called Multiplicative Weight Updates, or sometimes also called the "Hedge" algorithm; this relates to online decision-making. Later on, we'll also talk about its relation to zero-sum games.

What is online decision making? Suppose we have a decision that we're trying to make, and we have n experts that have conflicting opinions. Who do we trust, and which decision should we make? Here, in each successive trial, we know the history up until the current trial (that is, we know nothing in trial 1, but in trial 2, we know the results from trial 1, etc.).

Example 17.1

As an example, suppose we have to make a decision on whether the weather will be rainy on a given day. We have n experts, and each day we keep track of which experts were wrong, and keep a running "loss tally" for each expert (where 0 is a correct prediction and 1 is an incorrect prediction):

expert	day 1	loss tally	day 2	loss tally
1	1	1	1	2
2	1	1	0	1
3	0	0	1	1
4	0	0	0	0
5	1	1	1	2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Here, each day, we want to make an "online" decision; that is, we only know the results from previous days and the loss tallies up until now. After each day, the losses are revealed and we update the tallies.

Our goal is to find an algorithm that can make these decisions for us, utilizing this history that we have up until the current point in time.

Let us first introduce some notation that we'll use for the remainder of the lecture.

There are n experts, and we follow expert $i_t \in \{1, \dots, n\}$ on day $t \in \{1, \dots, T\}$, where T is the total number of days that we'll be doing this process on.

Each expert incurs a loss $\ell_i^{(t)}$ on day t ; this means that *our* total loss is $L := \sum_{t=1}^T \ell_{i_t}^{(t)}$. This sums over all days, taking the losses of the experts that we followed on that day. We want to minimize L .

Additionally, in general, there could be multiple actions that we could take (not just binary as we have in the previous example), and the losses can be some number $\ell_i^{(t)} \in [0, 1]$. Here, all that is needed is that the loss is bounded; we can

just rescale to $[0, 1]$ anyways.

Ideally, we want to compare $L = \sum_t \ell_{i_t}^{(t)}$ and $\sum_{t=1}^T \min_i \ell_i^{(t)}$; the latter case is if we magically know

This is not a realistic goal. Going back to the rain example, suppose we have two experts, where one expert always predicts that it does not rain, and one expert always predicts that it does rain. In the optimal case, we magically know whether it rains or not, and our loss is 0. However, more realistically, if it rains 50% of the time, no matter what we do, we incur a loss of approximately $\frac{T}{2}$; we expect to be wrong half of the time, as we can't possibly predict a uniformly random outcome.

As such, we will settle for something slightly weaker, i.e. compare our L to $\min_i \sum_{t=1}^T \ell_i^{(t)}$. That is, we compare our actual loss to the case where we look back and only follow one expert throughout. We further define

$$R := L - \min_i \sum_{t=1}^T \ell_i^{(t)} = L - L^*$$

as our “regret”. Here, L^* is what we’re comparing against. It turns out that we can have an algorithm that gives us very little regret for this definition.

17.2 Algorithms for Online Decision-making

Strategy 1: For all days t , we always follow expert 1; i.e. $i_t = 1$.

The claim is that our regret $R \leq T$; that is, we can always be wrong in every day. It isn't hard to come up with an example where $R = T$ (here we only give the losses on each day):

expert	day 1	day 2	day 3	...
1	1	1	1	...
2	0	0	0	...

In this case, we have $L = T$ and $L^* = 0$, meaning $R = L - L^* = T$.

Strategy 2: Follow the majority opinion.

The claim is that our regret $R \leq T$. Again, it isn't hard to come up with an example where $R = T$; we can have a bunch of experts that are wrong, and the very last expert that is always right:

expert	day 1	day 2	day 3	...
1	1	1	1	...
2	1	1	1	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	
$n-1$	1	1	1	...
n	0	0	0	...

In this case, we have $L = T$ and $L^* = 0$, meaning $R = L - L^* = T$.

Strategy 3: Each day, listen to a uniformly random expert.

(Just as a side-note, remember that in our worst-case analysis, we pretend that we have an adversary that can screw us over depending on what strategy we use. That is, the adversary knows everything about the algorithm and strategy, and can act accordingly to give us a worst-case outcome, which we're trying to analyze. However, one crucial limitation is that the adversary cannot possibly know what the output of randomness is—that is, random choices cannot be predicted by the adversary.)

The claim is that $\mathbb{E}[R] \geq (1 - \frac{1}{n})T$.

We will prove an upper bound on $\mathbb{E}[R]$; that is, $\mathbb{E}[R] \leq (1 - \frac{1}{n})T$. If we pick a uniformly random expert to follow, each day we have an expected loss of $\frac{1}{n} \sum_{i=1}^n \ell_i^{(t)}$. This means that our total expected loss is

$$\mathbb{E}[L] = \sum_{t=1}^T \left(\frac{1}{n} \sum_{i=1}^n \ell_i^{(t)} \right).$$

We further know that one of these i 's is actually the best person of the day, so we can split this up:

$$\mathbb{E}[L] = \sum_{t=1}^T \left(\min_i \ell_i^{(t)} + \frac{1}{n} \sum_{j \neq i} \ell_j^{(t)} \right).$$

Firstly, we can see that $\sum_{t=1}^T \min_i \ell_i^{(t)} \leq L^*$; this will usually be better than our target L^* , as we have the choice of which best expert to choose on each day. Further, since each loss is upper bounded by 1, and we have $n-1$ other experts in the inner summation, the second term $\frac{1}{n} \sum_{j \neq i} \ell_j^{(t)} \leq \frac{n-1}{n}$.

$$\mathbb{E}[L] \leq L^* + \sum_{t=1}^T \frac{n-1}{n} = L^* + \left(1 - \frac{1}{n}\right)T.$$

As such, we have the following upper bound for our expected regret $\mathbb{E}[R]$:

$$\mathbb{E}[R] = \mathbb{E}[L] - L^* \leq \left(1 - \frac{1}{n}\right)T.$$

The lower bound as claimed can also be shown through a similar probabilistic analysis, but what we can conclude is that this strategy can do at best a little bit less than T .

Strategy 4: Follow the expert who's been the best so far.

The claim is that $\mathbb{E}[R] \geq (1 - \frac{1}{n})T$, up to some small additive term.

We will give an example with three experts, but we can generalize this as well.

expert	$\ell^{(1)}$	tally	$\ell^{(2)}$	tally	$\ell^{(3)}$	tally	$\ell^{(4)}$	tally
1	0	0	1	1	0	1	0	1
2	$\frac{1}{3}$	$\frac{1}{3}$	0	$\frac{1}{3}$	1	$\frac{4}{3}$	0	$\frac{4}{3}$
3	$\frac{2}{3}$	$\frac{2}{3}$	0	$\frac{2}{3}$	0	$\frac{2}{3}$	1	$\frac{5}{3}$

Here, in the first day, we have no information, so WLOG suppose we choose expert 1. Then, on the next day, the adversary knows we will pick expert 1 to follow, so it will purposefully make expert 1 wrong, while all other experts are correct. The next day, the adversary knows we will pick expert 2 to follow, so it will adjust accordingly to give us the most loss. This can continue for all days, meaning that we're losing on every single day except for the first.

As such, our loss in this worst-case scenario is $T - 1$, with some small additive term to account for the first day that is insignificant with large T . This is because we have a loss of one every day except for the first.

One observation is that our first strategy would actually be better for this exact scenario; notice that each expert is mostly correct; any individual expert is only incorrect on $\frac{1}{n}$ of the days. This means that if we had used the first strategy, our expected loss would have been $\frac{T}{n}$, with some additive factor to account for the first day.

This means that we have an actual loss of $L \approx T$, with an optimal loss of $L^* \approx \frac{T}{n}$ with our first strategy. Our regret is thus $R = L - L^* \approx (1 - \frac{1}{n})T$.

Strategy 5: (Multiplicative weights/Hedge algorithm) Use both history *and* randomness.

In this strategy, on each day we have a probability distribution $\tilde{\mathbf{x}}^{(t)}$ on the experts, and on each day, we update this probability based on their performance.

Here, our expected loss on day t is

$$L_t = \langle \tilde{\mathbf{x}}^{(t)}, \tilde{\ell}^{(t)} \rangle = \sum_{i=1}^n x_i^{(t)} \cdot \ell_i^{(t)}.$$

As such, our total loss is $L = \sum_{t=1}^T L_t$; note that this takes the expectation into account (i.e. this is our expected loss).

Definition 17.2: Multiplicative Weights/Hedge Algorithm

This algorithm takes in an input parameter $\varepsilon \in (0, \frac{1}{2}]$. We'll define weights $w_i^{(t)}$, such that their initial values $w_i^{(1)} = 1$ for all $i \in \{1, \dots, n\}$. This means that our probability distribution is

$$x_i^{(t)} = \frac{w_i^{(t)}}{\sum_{j=1}^n w_j^{(t)}}.$$

Here, we'll define $W^{(t)} = \sum_{j=1}^n w_j^{(t)}$ as the total weight on day t . To update these weights, we have

$$w_i^{(t+1)} = w_i^{(t)} \cdot (1 - \varepsilon)^{\ell_i^{(t)}}.$$

Intuitively, what does this mean? In the extreme case of a binary loss, the people that are right will not have their weights updated. However, the people that are wrong will have their weights exponentially dampened at a rate of $1 - \varepsilon$ per day; each day we'd be multiply the weight by a factor of $1 - \varepsilon$.

With non-binary losses, the dampening varies based on how bad the loss is, but the same idea holds; we exponentially penalize the people that are wrong, and do nothing to the people that are correct.

To analyze this algorithm, we will use two main lemmas:

Lemma 17.3

The weight at the end of day T , $W^{(T+1)}$, is at most $(1 - \varepsilon)^{L^*}$. (Here we use $T + 1$ because $W^{(t)}$ is the weight at the beginning of day t .)

Proof. We know that

$$W^{(T+1)} = \sum_{i=1}^n w_i^{(T+1)} \geq w_{i^*}^{(T+1)},$$

where here i^* is the best expert, i.e. it's the expert that defines L^* .

We further know that

$$w_{i^*}^{(T+1)} = \prod_{t=1}^T (1 - \varepsilon)^{\ell_{i^*}^{(t)}} = (1 - \varepsilon)^{\sum_{t=1}^T \ell_{i^*}^{(t)}} = (1 - \varepsilon)^{L^*}.$$

□

Lemma 17.4

$$W^{(T+1)} \leq n \cdot \prod_{t=1}^T (1 - \varepsilon L_t).$$

Proof. We will show that in general, $W^{(t+1)} \leq W^{(t)}(1 - \varepsilon L_t)$. If we had this, then by induction $W^{(T+1)} \leq W^{(1)} \cdot \prod_{t=1}^T (1 - \varepsilon L_t)$, where $W^{(1)} = n$ on the first day.

So, to show the first inequality, we know that

$$W^{(t+1)} = \sum_{i=1}^n w_i^{(t+1)} = \sum_{i=1}^n w_i^{(t)} \cdot (1 - \varepsilon)^{\ell_i^{(t)}}.$$

Here, we can bound $(1 - \varepsilon)^{\ell_i^{(t)}} \leq (1 - \varepsilon \ell_i^{(t)})$, meaning we have

$$W^{(t+1)} \leq \sum_{i=1}^n w_i^{(t)} \cdot (1 - \varepsilon \ell_i^{(t)}).$$

Knowing that $w_i^{(t)} = \sum_i x_i^{(t)} W^{(t)}$, we have that

$$W^{(T+1)} \leq \sum_{i=1}^n x_i^{(t)} W^{(t)} \cdot (1 - \varepsilon \ell_i^{(t)}) = W^{(t)} \left(\underbrace{\sum_{i=1}^n x_i^{(t)}}_1 - \varepsilon \underbrace{\sum_{i=1}^n x_i^{(t)} \ell_i^{(t)}}_{L_t} \right) = W^{(t)} (1 - \varepsilon L_t).$$

The last simplification makes use of the fact that $\mathbf{x}^{(t)}$ is a probability distribution (and hence its elements sum to 1), and the definition of the expected loss L_t .

Since we've now proved our initial claim, the original lemma is proven as well. $\square$

Theorem 17.5

Hedge(ε) achieves $\mathbb{E}[R] \leq \varepsilon \cdot T + \frac{\ln n}{\varepsilon}$. If we choose $\varepsilon = \sqrt{\frac{\ln n}{T}}$ to equate these terms, we have $\mathbb{E}[R] \leq 2\sqrt{T \ln n}$.

Proof. We will use the two prior lemmas to prove this theorem.

The lemmas give us an upper and a lower bound for W_T , so we can compare the two sides; now taking the natural log of both sides, we have

$$(1 - \varepsilon)^{L^*} \leq n \cdot \prod_{t=1}^T (1 - \varepsilon L_t)$$

$$L^* \ln(1 - \varepsilon) \leq \ln n + \sum_{t=1}^T \ln(1 - \varepsilon L_t)$$

We will now make use of the fact that $\forall z \in [0, \frac{1}{2}]$, $-z - z^2 \leq \ln(1 - z) \leq -z$; this can be derived by taking the first few terms of the Taylor expansion.

This means that we have

$$L^* (-\varepsilon - \varepsilon^2) \leq \ln n - \varepsilon \sum_{t=1}^T L_t$$

$$L^* (-1 - \varepsilon) \leq \frac{\ln n}{\varepsilon} - \sum_{t=1}^T L_t$$

$$\sum_{t=1}^T L_t - L^* \leq \frac{\ln n}{\varepsilon} + \varepsilon L^*$$

We know that our loss $L = \sum_{t=1}^T L_t$, and we can bound $L^* \leq T$, so this means that we have

$$R = L - L^* \leq \frac{\ln n}{\varepsilon} + \varepsilon T,$$

as desired. $\square$

17.3 Connection to Zero Sum Games

We used strong duality last lecture to prove that if we have two players in a zero sum game, the expected payoff is the same for each optimization problem (one for each player), which were duals.

It turns out that we can just use weak duality combined with the main theorem of the Hedge algorithm; this also proves that the expected payoff is the same for both players. Further, this actually gives us an algorithm to find this pair of strategies.

The row player and the column player both start with a uniform mixed strategy. On day t , the row player is revealed a loss vector $\ell^{(t)} = -\mathbf{A}\tilde{\mathbf{y}}^{(t)}$, and the column player is revealed a loss vector $\ell^{(t)} = \mathbf{A}^T \tilde{\mathbf{x}}^{(t)}$. That is, we rig the loss vectors such that the resulting payoffs are exactly the same as the Hedge algorithm.

If we run multiple iterations of this game and modify the probability distributions for each player, we'd end up converging to the same expected payoff (by weak duality).

10/28/2021

Lecture 18

Reductions, Bipartite Matching

18.1 Reductions

Definition 18.1: Reduction

Suppose we have two computational problems A and B . We say that “ A reduces to B ”, i.e. $A \rightarrow B$, if an efficient algorithm for B means that there is an efficient algorithm for A .

The good thing here is that if we know how to solve B efficiently, then we can solve A efficiently.

The bad thing is that if we can prove that A has no efficient algorithm, then B must not have an efficient algorithm either (because then we could just use the algorithm for B to solve A).

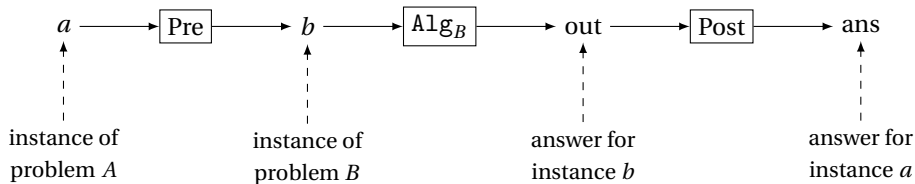
Hence, we can use reductions to get efficient algorithms and also to prove no efficient algorithms exist.

There are a few types of reductions. For example,

- Turing reductions
- Cook reductions
- Many-to-one reductions
- Log-space reductions
- etc.

For today, we will mostly be talking about Cook reductions. Specifically, we're dealing with reductions that take polynomial time. That is, if we know a polynomial time algorithm Alg_B that solves B , then we can call it (possibly multiple times) inside a solution to A . This means that A can be solved in polynomial time as well.

A possible Cook reduction could be as follows:



A more general reduction could make multiple calls to Alg_B , but this is the general process. We take an instance of problem A (i.e. a specific case of the problem A), do some pre-processing, and feed it in as an input for an instance of problem B . Utilizing Alg_B to get the answer for this instance, we can then do some post-processing to turn that into an answer for A .

We'll look at a few examples today:

- Maximal Bipartite Matching $\rightarrow$ Max Flow
- Any problem $\in P \rightarrow$ Circuit Value Problem $\rightarrow$ Linear Programming